

Trajectory-Aware Stability Supervision for Neural Architecture Search Using a Phi Controller Across 2, 10, and 100 Class Counts

Shawn Barnicle
Barnicle AI Systems
Chicago, Illinois
`ShawnBarnicle.ai@gmail.com`

May 2026

Abstract

We present a controller for supervising neural network training during architecture search that combines a trajectory-aware termination rule with a normalized thermodynamic stability metric. The stability metric is $\Phi = I \times \rho - \alpha \times S_{\text{norm}}$, where I is identity normalized against the random baseline, ρ is temporal coherence of the accuracy history, S_{norm} is confusion matrix entropy normalized for class count, and $\alpha = 0.1$. The trajectory-aware rule terminates training only when the best Φ achieved from the start of training remains below a viability threshold and improvement has stalled, distinguishing slow-starting viable architectures from genuinely non-viable ones. In a direct comparison on 30 architectures, the controller produced 0 false kills while patience-based early stopping produced 20 false kills (16.7% kill precision) on the same architecture set. Across 660 total architectures spanning multilayer perceptrons and convolutional networks on 2-class, 10-class, and 100-class problems, the controller achieved 2 false kills (99.7% kill precision). A threshold of 0.25 applies to 2 through 10 class problems and 100-class multilayer perceptrons. A separately calibrated threshold of 0.084 applies to 100-class convolutional networks. An intermediate hypothesis tested during development, that a fixed relative-performance constant of 0.56 could serve as a class-count-agnostic threshold, failed on 2-class data and motivated the formula normalization approach instead. The controller is designed as a conservative kill filter: it accepts false passes to eliminate false kills, and its primary validated property is kill precision rather than balanced binary classification accuracy.

1 Introduction

Neural architecture search and hyperparameter optimization require evaluating large numbers of candidate architectures. The standard workflow trains each candidate for many epochs before determining whether it is viable, meaning that a search over one hundred architectures at fifty epochs per candidate requires one hundred full training runs totaling five thousand training epochs before the viable set is known. A meaningful fraction of those runs fail: the architecture never reaches useful performance on the target task regardless of how long training continues.

Existing approaches to reducing this cost fall into two broad categories. Early stopping methods monitor validation performance over training and halt when improvement plateaus [5]. Methods such as Successive Halving and Hyperband extend this idea by allocating progressively more resources to promising configurations across multiple rounds [6, 7]. These methods share a structural limitation: they allocate resources using partial-training performance rather than explicitly tracking best

progress from the start of training. An architecture that improves slowly but steadily, or that plateaus temporarily before recovering, will often be terminated prematurely because its recent improvement does not meet the evaluation criterion.

A second category of methods attempts to predict viability before training is complete, either by extrapolating learning curves from early epochs [8, 9] or by computing parameter-space proxies at initialization [10, 11]. Learning curve extrapolation requires multiple epochs to fit a reliable curve and operates on scalar performance trajectories without examining the internal structure of model behavior across classes. Parameter-space proxies do not require labeled data and are computationally inexpensive, but they operate in weight space rather than behavioral space and do not directly measure whether the network has begun forming coherent class-conditional predictions.

This paper takes a different approach. Rather than evaluating recent improvement or parameter statistics, we supervise training using a stability metric that measures three properties simultaneously: how far the architecture’s predictions are above the random baseline (identity), how consistent the accuracy history is over time (coherence), and how organized the confusion matrix is relative to its maximum possible entropy (entropy). The metric is computed at each epoch and the controller tracks the best value achieved from training start. Termination occurs only when the best value has remained below the viability threshold and no meaningful improvement has been observed for a patience window.

The method connects to prior work using confusion matrix statistics for behavioral assessment [15, 16, 17]. Prior work in this line used diagonal strength and correlation between confusion matrices to predict training efficiency and architectural viability after one epoch. The present work extends the framework to ongoing training supervision and introduces normalized identity and normalized entropy components that make the metric applicable across class counts ranging from 2-class binary classification to 100-class image recognition.

The contributions of this paper are as follows. We identify a trajectory-aware termination rule that eliminates the slow-starter false kills that characterize patience-based early stopping, confirmed in a direct 30-architecture comparison. We introduce a normalized stability metric that operates across 2-class, 10-class, and 100-class problems under one formula with two validated class-range thresholds. We document a failed intermediate hypothesis, that a fixed relative-performance constant transfers the 10-class threshold to other class counts, and show that formula normalization rather than threshold adjustment is the correct generalization mechanism. We characterize the controller as a conservative kill filter whose validated property is kill precision rather than balanced viability classification accuracy. And we report the boundary condition at 100-class convolutional networks where the formula continues to hold but a separately calibrated threshold is required.

To our knowledge, prior work does not combine trajectory-aware best-progress termination logic with a normalized thermodynamic stability metric for architecture search supervision across this range of class counts.

2 The Method

2.1 The Stability Metric

Let CM_t denote the confusion matrix produced by the network at training epoch t , evaluated on a held-out validation set. For a classification task with N classes, CM_t is an $N \times N$ array where element (i, j) contains the count of samples with true label i and predicted label j .

The formation score F_t is the diagonal strength of this matrix:

$$F_t = \frac{\text{tr}(\text{CM}_t)}{\sum_{i,j} (\text{CM}_t)_{ij}} \quad (1)$$

In standard single-label classification, F_t is numerically equal to classification accuracy at epoch t .

The identity component I_t normalizes the formation score against the random baseline for N classes:

$$I_t = \max\left(0, \frac{F_t - \frac{1}{N}}{1 - \frac{1}{N}}\right) \quad (2)$$

This maps random-or-worse performance to 0 and perfect performance to 1, so that equivalent levels of task competence produce comparable identity values across 2-class, 10-class, and 100-class problems.

The temporal coherence ρ_t measures how consistent the accuracy history has been through epoch t :

$$\rho_t = \text{corr}(F_1, \dots, F_{t-1}, F_2, \dots, F_t) \quad (3)$$

This is the lag-1 Pearson autocorrelation of the accuracy sequence. Architectures with stable, monotonically improving training produce ρ_t close to 1. Erratic or non-improving architectures produce lower values. For accuracy histories shorter than three values, ρ_t is set to 1.0 to avoid unstable autocorrelation estimates during the earliest epochs.

The normalized entropy $S_{\text{norm},t}$ measures how disorganized the confusion matrix is relative to its maximum possible disorder:

$$S_{\text{norm},t} = \frac{H(\text{CM}_t)}{\log_2(N^2)} \quad (4)$$

where $H(\text{CM}_t)$ is the Shannon entropy of the normalized confusion matrix treated as a probability distribution, and $\log_2(N^2)$ is used as an upper-bound normalization constant for the entropy of an $N \times N$ confusion matrix. This normalization constrains $S_{\text{norm},t}$ to $[0, 1]$ regardless of class count.

Without this normalization, the raw entropy penalty becomes several times harsher as class count rises: the maximum entropy is $\log_2(4) = 2.0$ bits for 2-class, $\log_2(100) = 6.64$ bits for 10-class, and $\log_2(10000) = 13.29$ bits for 100-class problems, corresponding to scaling ratios of approximately 3.3 and 6.6 relative to the 2-class baseline. A formula calibrated with raw entropy on 2-class data will systematically underestimate stability on 10-class and 100-class data.

The stability metric combines these three components:

$$\Phi_t = I_t \times \rho_t - \alpha \times S_{\text{norm},t} \quad (5)$$

where $\alpha = 0.1$ is a coupling constant held fixed across all experiments.

2.2 Viability Classification

The primary viability rule is:

- $\Phi_t \geq \Phi_c$: the architecture is predicted viable and training continues.
- $\Phi_t < \Phi_c$: the architecture may be non-viable, subject to the trajectory-aware termination conditions described below.

Two validated thresholds apply depending on the classification setting:

- $\Phi_c = 0.25$ for 2-class and 10-class problems, and for 100-class multilayer perceptrons.
- $\Phi_c = 0.084$ for 100-class convolutional networks, calibrated by post-hoc threshold grid analysis on the completed 100-architecture CIFAR-100 validation run described in the Results section.

2.3 Trajectory-Aware Termination

The controller does not terminate an architecture the first time Φ_t falls below the threshold. Instead, it tracks the best Φ value achieved from the start of training:

$$\Phi_t^* = \max_{s \leq t} \Phi_s \quad (6)$$

Meaningful improvement is defined as an increase in Φ_t^* greater than 0.05. If meaningful improvement occurs, the controller updates the best recorded value and resets the no-improvement counter. Otherwise, the number of epochs without improvement is incremented.

Termination occurs only when all three of the following conditions hold simultaneously in the validated controller configuration:

1. At least 4 epochs have elapsed.
2. Φ_t^* has remained below the viability threshold throughout.
3. The number of epochs without meaningful improvement has reached 2.

This rule distinguishes slow-starting viable architectures, which improve gradually and may remain below threshold for several epochs before crossing it, from architectures that make no meaningful progress from the start of training.

2.4 Controller Design Objective

The controller is designed as a conservative kill filter rather than a balanced binary classifier. It accepts false passes (allowing non-viable architectures to continue training unnecessarily) in order to eliminate false kills (terminating architectures that would have become viable). In NAS settings, the operational cost of a false kill is much higher than the cost of a false pass: a false kill discards a viable architecture and the compute already invested in it, while a false pass only incurs extra training time. The controller’s primary validated property is kill precision, not overall binary classification accuracy.

3 Experiments

3.1 Datasets

All experiments used real labeled image classification data with no synthetic augmentation. The Wisconsin Breast Cancer dataset [1] provided the 2-class setting (398 training samples, 171 test samples). MNIST [2] and Fashion-MNIST [3] provided 10-class grayscale image settings. For MNIST we used both a 7,000-sample subset and the full 70,000-sample dataset across separate experiments. CIFAR-10 [4] provided the 10-class color image setting (5,000 training, 1,000 test samples per experiment). CIFAR-100 [4] provided the 100-class settings for both multilayer perceptrons and convolutional networks. All datasets were accessed from publicly available sources using standard loading utilities. For all experiments, a stratified held-out validation split was used for computing Φ_t at each epoch. Test set accuracy is reported as the ground truth viability outcome.

3.2 Architectures

Two architecture families were evaluated. Multilayer perceptrons use fully connected layers with ReLU activations. Convolutional networks use sequences of convolutional and pooling layers followed by a classification head.

MLP architectures were generated by random sampling of layer counts and widths, including standard configurations and intentionally capacity-limited bottleneck designs. In total, 445 MLP architectures were evaluated across Breast Cancer (n=35), MNIST (n=305), Fashion-MNIST (n=100), and CIFAR-100 (n=5). CNN architectures were evaluated on CIFAR-10 (n=115) and CIFAR-100 (n=100). The CIFAR-100 CNN validation was run on Google Colab using a T4 GPU and is documented as such.

3.3 Procedure

For each architecture, the procedure was as follows. The network was initialized with random weights and trained using standard gradient-based optimization and backpropagation. At each epoch, Φ_t was computed from the validation set confusion matrix. The controller tracked Φ_t^* and applied the trajectory-aware termination rule. Training continued to a full budget if the architecture was not terminated: 30 epochs for most MLP settings, 50 epochs for full-scale MNIST and CIFAR-10 CNN, and 30 epochs for CIFAR-100 CNN. After training, ground truth viability was assigned based on whether the architecture reached a class-appropriate accuracy threshold: 60% for 2-class and 10-class problems, 15% for 100-class multilayer perceptron validation, and 20% for 100-class convolutional network validation. These thresholds were chosen to represent performance substantially above the random baseline in each setting while preserving a conservative definition of viability for controller evaluation.

Experiments were implemented in Keras/TensorFlow with confusion matrices computed using scikit-learn. Random seeds were set before each experiment. No synthetic data was used at any stage. Results were saved to timestamped output files at experiment completion.

3.4 The 0.56 Hypothesis and Its Failure

Before arriving at the normalized Φ formula, we tested an intermediate hypothesis derived from the predecessor one-epoch viability method based on confusion matrix diagonal strength alone. In that

earlier formulation, the 10-class viability boundary corresponded to an accuracy of approximately 60.84%, which represents roughly 56.5% relative performance above the 10% random baseline:

$$\frac{0.6084 - 0.10}{1 - 0.10} \approx 0.565 \quad (7)$$

We hypothesized that this 0.56 relative-performance value might serve as a class-count-agnostic viability criterion, allowing the required accuracy threshold to be computed for any class count as:

$$\text{viable_acc} = \frac{1}{N} + 0.56 \times \left(1 - \frac{1}{N}\right) \quad (8)$$

For 2-class problems this predicts a viability threshold of 78.0%. Tested on the Breast Cancer 2-class dataset, this hypothesis produced one false kill: a viable architecture was terminated at 62.6% accuracy but subsequently reached 95%+ with full training. This single counterexample falsified the hypothesis. The failure established that cross-class-count generalization requires normalizing the formula components rather than adjusting the threshold, which motivated the normalized identity and normalized entropy design described in Section 2.

3.5 Head-to-Head Comparison With Early Stopping

We compared the controller against standard patience-based early stopping on a matched set of 30 MNIST MLP architectures. Both methods ran on identical architectures with identical random seeds. Early stopping used a patience of 5 epochs, which is a standard default in common deep learning frameworks. The controller used the trajectory-aware termination rule with the validated defaults described in Section 2. Ground truth viability was defined as reaching 60% accuracy. For the 30-architecture head-to-head comparison, both methods were evaluated under a matched 20-epoch maximum training budget per architecture.

4 Results

4.1 Head-to-Head Comparison

Table 1 reports the direct comparison between the controller and early stopping on 30 architectures.

Table 1: Direct comparison of the Phi controller against early stopping with patience=5 on 30 matched MNIST MLP architectures. Both methods used identical architectures and random seeds. Ground truth viability defined as reaching 60% accuracy.

Method	False Kills	Kill Precision	Total Epochs
Phi controller	0	100%	540
Early stopping (patience=5)	20	16.7%	449

The controller produced 0 false kills while early stopping produced 20 false kills on the same 30 architectures. The controller achieved 100% kill precision (4/4 correct kills) while early stopping achieved 16.7% kill precision (4/24 correct kills). Early stopping terminated 24 of the 30 architectures early, of which 20 were viable architectures that would have reached 60% accuracy given full training. The controller used 540 total epochs versus 449 for early stopping, a difference of 91 epochs, because

it allowed slow starters to continue rather than terminating them during temporary plateaus. Average accuracy loss from the controller was 0.34%. Average accuracy loss from early stopping was 0.38%.

4.2 MLP Validation

Table 2 reports validation results for all MLP experiments across 2-class, 10-class, and 100-class settings.

Table 2: MLP validation results across all class counts. Threshold 0.25 applied throughout. Ground truth viability thresholds: 60% for 2-class and 10-class, 15% for 100-class MLPs.

Dataset	Classes	Architectures	False Kills	Kill Precision
Breast Cancer	2	35	0	100%
MNIST (subset)	10	105	0	100%
MNIST (full 70K)	10	100	0	100%
MNIST (bottleneck)	10	100	0	100%
Fashion-MNIST	10	100	0	100%
CIFAR-100 MLPs	100	5	0	100%
MLP Total		445	0	100%

Across 445 MLP architectures spanning 2-class, 10-class, and 100-class problems, the controller produced 0 false kills and 100% kill precision at threshold 0.25. This result holds across a broad range of MLP configurations, including intentionally capacity-limited bottleneck architectures, architectures trained on the full 70,000-sample MNIST dataset for 50 epochs, and 100-class CIFAR-100 problems. The 100-class MLP result (5 architectures, 0 false kills) is the least comprehensive of the MLP tests and should be interpreted accordingly.

4.3 10-Class CNN Validation

Table 3 reports results for 10-class convolutional network validation on CIFAR-10.

Table 3: 10-class CNN validation on CIFAR-10. Threshold 0.25. Ground truth viability defined as reaching 60% accuracy. Two false kills occurred at edge cases with accuracies of 60.6% and 61.5%.

Test	Classes	Architectures	False Kills	Kill Precision
CIFAR-10 CNN (15 arch)	10	15	0	100%
CIFAR-10 CNN (100 arch)	10	100	2	98.0%
10-Class CNN Total		115	2	98.3%

Across 115 10-class CNN architectures, the controller produced 2 false kills and 98.3% kill precision. Both false kills were edge cases with final accuracies of 60.6% and 61.5%, marginally above the 60% viability boundary. These architectures had high confusion matrix entropy at the time of termination, which caused the formula to underestimate their stability. This is the method’s characteristic failure mode on barely-viable architectures near the viability boundary.

4.4 100-Class CNN Validation and Threshold Calibration

Applying the 0.25 threshold to 100-class convolutional networks produced excessive false kills in initial testing. Viable 100-class CNN architectures achieving 20% to 28% accuracy (20 to 28 times better than the 1% random baseline) produced Φ values clustering between 0.08 and 0.15, well below the 0.25 threshold. The formula is structurally correct for this setting, but the viable population occupies a different region of the Φ space than it does at lower class counts.

To find an appropriate threshold, we first ran a full 100-architecture CIFAR-100 CNN validation at a provisional threshold of 0.10. This produced 1 false kill: architecture Seed 83 (configuration [64, 64] with a 256-unit fully connected head and batch normalization) achieved 20.6% accuracy with $\Phi = 0.084$ and was incorrectly terminated. We then performed a post-hoc threshold grid analysis by sweeping threshold values from 0.060 to 0.120 using the Φ values already computed during the validation run. Table 4 reports selected results from this analysis.

Table 4: Threshold grid analysis for 100-class CNN validation on 100 CIFAR-100 architectures. False kills and false passes reported at selected threshold values. Threshold 0.084 achieves 0 false kills.

Threshold	False Kills	False Passes	Notes
0.060	0	28	
0.080	0	21	
0.084	0	19	Selected
0.085	1	17	
0.100	1	8	Initial provisional
0.120	3	0	

Threshold 0.084 achieves 0 false kills with 19 false passes and was selected as the validated threshold for 100-class CNN settings. The tradeoff at this threshold is 19 false passes compared to 8 at the provisional 0.10, which is acceptable because false passes only cause extra training time without discarding viable architectures. The edge case (Seed 83, $\Phi = 0.084$) sits exactly at the selected threshold boundary and passes under it.

One important limitation of this calibration: the threshold 0.084 was derived from the same CIFAR-100 CNN result set used for validation rather than from an independent held-out calibration set. This makes the result valid as a confirmed internal finding but weaker than a prospective threshold pre-registration would be.

4.5 Hyperparameter Robustness

We evaluated the controller across standard training hyperparameter variations on a single MNIST 10-class MLP architecture ([128, 64]). Table 5 summarizes the results.

Optimizers tested were Adam, SGD, SGD with momentum, RMSprop, and Adagrad, with final accuracies ranging from 89.1% to 91.3% and Φ values from 0.588 to 0.783. Learning rates tested ranged from 0.0001 to 0.1, with final accuracies from 83.2% to 91.2%. Batch sizes tested ranged from 16 to 512, with final accuracies from 90.7% to 92.4%.

An important boundary on this result: all 18 tested configurations were viable architectures. The test establishes that the controller correctly passes viable architectures across these hyperparameter settings but does not establish whether it correctly kills non-viable architectures across the same

Table 5: Hyperparameter robustness on a single MNIST 10-class MLP. All 18 tested configurations are viable architectures. 0 false kills across all configurations.

Dimension	Configurations Tested	False Kills	Kill Precision
Optimizer	5	0	100%
Learning rate	7	0	100%
Batch size	6	0	100%
Total	18	0	100%

range of settings. The test was conducted on one architecture and one dataset. Whether the result extends to other architecture families or datasets is not established.

4.6 Summary

Table 6 summarizes all validation results.

Table 6: Summary of all validation experiments. Total across 660 architectures: 2 false kills, 99.7% kill precision.

Setting	Threshold	Architectures	False Kills	Kill Precision
MLPs (2, 10, 100 class)	0.25	445	0	100%
CNNs 10-class	0.25	115	2	98.3%
CNNs 100-class	0.084	100	0	100%
Total		660	2	99.7%

The controller is not a balanced binary classifier. In several experiments, the number of false passes substantially exceeded the number of correct kills. In the MNIST 100-architecture random test, 53 of 100 predictions were correct overall, with 0 false kills and 47 false passes. In the bottleneck architecture test, there were 0 false kills alongside 10 false passes. This asymmetric error structure is consistent with the controller’s design objective: it is optimized to minimize false kills rather than to maximize overall binary accuracy.

5 Discussion

5.1 Why Trajectory-Aware Logic Outperforms Patience-Based Stopping

The head-to-head comparison result, 0 false kills versus 20 on the same 30 architectures, is explained by a single structural difference in the decision logic. Patience-based early stopping evaluates whether the architecture has improved recently. The trajectory-aware controller evaluates whether the architecture has improved at all from the start of training. These are different questions, and the difference matters for slow-starting architectures.

An architecture that reaches $\Phi_t = 0.15$ at epoch 5, stalls at 0.16 for several epochs, then climbs to 0.32 by epoch 20 will be terminated by patience-based stopping because it fails the recent-improvement criterion during the stall. The trajectory-aware controller tracks Φ_t^* and sees that the best value is still improving overall, so it allows training to continue. This distinction is the mechanism behind the false-kill reduction observed in the head-to-head comparison. The result

shows that best-progress logic can protect slow-starting viable architectures that would be discarded by patience-based stopping.

The cost of the trajectory-aware approach is fewer total epoch savings. The controller used 540 epochs versus 449 for early stopping on the 30-architecture comparison, a difference of 91 epochs. Early stopping achieved greater compute reduction, but at the cost of destroying 20 viable architectures in the process. Whether that tradeoff is acceptable depends on the application. In settings where viable architectures are rare or expensive to rediscover, the controller’s design is preferable.

5.2 What the Formula Measures and What It Does Not

The stability metric $\Phi_t = I_t \times \rho_t - \alpha \times S_{\text{norm},t}$ combines three signals: how far above random the architecture’s predictions are, how temporally consistent the accuracy history is, and how organized the confusion matrix is relative to maximum entropy. An architecture that is improving on all three dimensions simultaneously will produce rising Φ_t .

The metric does not measure final accuracy directly, and it does not predict how fast a viable architecture will converge. It measures whether the architecture is in the process of forming coherent class-conditional predictions. An architecture with high Φ_t has begun sorting inputs into meaningful categories with some consistency over time. An architecture with persistently low Φ_t has not.

The conservative design of the controller follows from this interpretation. Because the metric measures an ongoing process rather than a final state, the right response to a low value is to check whether the process is still developing before terminating it. The trajectory-aware rule implements this check.

5.3 The Two-Threshold Result and What It Means

The formula required two thresholds rather than one. The 0.25 threshold works for 2-class through 10-class problems and for 100-class multilayer perceptrons. The 0.084 threshold is required for 100-class convolutional networks. The correct interpretation is that the formula generalizes more broadly than any single threshold does.

The reason the thresholds differ is that the Φ values of viable 100-class CNN architectures cluster in a lower range than those of viable architectures at other class counts. A 100-class CNN achieving 25% accuracy (25 times the 1% random baseline) is a genuinely viable architecture, but its identity, coherence, and entropy values combine to produce Φ near 0.10 rather than near 0.30. The normalization in the formula makes the components comparable across class counts, but it does not make the absolute Φ values identical across fundamentally different learning regimes.

The practical implication is that deploying this controller on a new class count or architecture family should include a threshold calibration step using the systematic grid analysis procedure described in Section 4.4. That procedure requires no additional training and can be performed on an existing validation run.

5.4 The Conservative Kill Filter Design

Several experiments produced many false passes alongside few or zero false kills. In the MNIST 100-architecture random test, the controller passed 47 non-viable architectures while killing none of the viable ones. This is not a deficiency in the method. It reflects a deliberate design choice that the cost of a false kill is substantially higher than the cost of a false pass in NAS settings.

The distinction matters for how the results should be read. The controller’s headline number is 99.7% kill precision across 660 architectures, not overall binary accuracy. A reader who evaluates this method on overall classification accuracy will find the number unimpressive in many individual tests. A reader who evaluates it on the specific question of whether viable architectures are ever discarded will find the result much stronger.

6 Limitations

The validation in this paper is bounded to the tested architecture families, datasets, and class counts. Several limitations follow directly from this scope.

The 100-class CNN threshold of 0.084 was derived from the same validation run it was applied to, not from an independent held-out calibration set. This is a real limitation of the threshold result. Whether 0.084 generalizes to other 100-class CNN datasets or architecture families has not been tested.

The 100-class MLP validation used only 5 architectures. This is the weakest individual result in the paper and should not be given the same weight as the larger MLP tests.

The hyperparameter robustness tests covered only viable architectures on one dataset and one architecture. Whether the controller correctly kills non-viable architectures across different optimizer and learning rate settings has not been verified.

The method has not been tested on modern architecture families such as residual networks or attention-based architectures. Whether the formula, thresholds, and termination logic transfer to these families requires additional validation.

The trajectory-aware termination rule was validated under one set of controller defaults. Whether those defaults are appropriate for other training regimes with different convergence timescales has not been studied.

Finally, the ground truth viability thresholds used in this paper (60% for 2 and 10-class, 15% for 100-class MLPs, 20% for 100-class CNNs) are operational choices, not universal definitions of viability. Results might differ under other definitions.

7 Related Work

7.1 Early Stopping and Resource Schedulers

Early stopping with patience-based criteria is the standard baseline for terminating underperforming training runs [5]. Successive Halving [7] and Hyperband [6] extend early stopping to multi-fidelity settings, allocating more resources to configurations that survive multiple evaluation rounds. These methods allocate resources using partial-training performance and multi-stage evaluation rather than explicitly tracking best progress from the start of training. As a result, they do not implement the trajectory-aware logic used here to protect slow-starting viable architectures. The present controller addresses this limitation directly through the trajectory-aware best-progress rule.

7.2 Learning Curve Extrapolation

Domhan et al. [8] proposed fitting parametric learning curves to early training data to project final performance. Baker et al. [9] incorporated learning curve features into a reinforcement learning

based NAS framework. These methods require multiple epochs to fit a reliable curve, operate on scalar accuracy trajectories, and do not examine the class-conditional structure of model behavior. The present method uses per-epoch confusion matrix statistics rather than scalar accuracy history as the primary signal.

7.3 Zero-Cost Proxies

Mellor et al. [10] proposed an architecture proxy based on activation correlations at initialization. Abdelfattah et al. [11] evaluated a range of zero-cost proxies including gradient norms and weight statistics. These methods require no labeled data and operate before training begins. The present method differs in that it supervises ongoing training using behavioral outputs rather than parameter statistics, and it is intended to work alongside training rather than replace it.

7.4 Neural Architecture Search

Zoph and Le [12] introduced reinforcement learning based NAS. Real et al. [13] proposed evolutionary architecture search. Liu et al. [14] introduced differentiable architecture search. These methods generate and evaluate candidate architectures but rely on early stopping or fixed epoch budgets to manage evaluation cost. The present controller can be used within any of these search frameworks as a supervision layer that reduces the cost of evaluating non-viable candidates.

7.5 Confusion Matrix Methods for Behavioral Assessment

Prior work in this portfolio used confusion matrix statistics to measure behavioral stability and predict training outcomes. The task-identity method [15] uses Pearson correlation between confusion matrices at two time points to detect behavioral drift. The formation detection method [16] uses correlation between an epoch-one confusion matrix and a mature checkpoint to predict training efficiency. The phase transition method [17] uses single-epoch confusion matrix diagonal strength to predict architectural viability before training is well underway. The present method extends this line by applying the full three-component stability metric as a continuous training supervisor rather than as a one-time early-epoch predictor.

8 Conclusion

We presented a controller for supervising neural network training during architecture search that combines a trajectory-aware termination rule with a normalized thermodynamic stability metric. The core operational contribution is the termination logic: tracking the best Φ achieved from the start of training rather than evaluating recent improvement substantially reduces the false kills on slow-starting architectures that characterize patience-based early stopping.

In a direct comparison on 30 architectures, the controller produced 0 false kills while early stopping with patience=5 produced 20 false kills on the same architecture set. Across 660 total architectures spanning multilayer perceptrons and convolutional networks on 2-class, 10-class, and 100-class problems, the controller achieved 2 false kills and 99.7% kill precision.

The normalized stability metric extends the behavioral assessment framework to multiple class counts by normalizing both identity and entropy against class-count-dependent baselines. A single threshold of 0.25 applies to 2-class through 10-class problems and to 100-class multilayer perceptrons.

A separately calibrated threshold of 0.084 applies to 100-class convolutional networks in the tested setting, where the viable architecture population occupies a lower region of the Φ space.

The controller is designed as a conservative kill filter. It accepts false passes to minimize false kills, and its primary validated property is kill precision rather than balanced binary classification accuracy. The results are bounded to the tested architecture families and datasets. Extension to modern architecture families and other task types requires additional validation.

Code Availability

All validation code, experiment scripts, and timestamped result outputs are available at <https://github.com/Wise314/phi-controller>.

References

- [1] Wolberg, W.H., Street, W.N., and Mangasarian, O.L. (1995). Machine learning techniques to diagnose breast cancer from fine-needle aspirates. *Cancer Letters*, 77(2), 163–171.
- [2] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [3] Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*.
- [4] Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Technical report, University of Toronto.
- [5] Prechelt, L. (1998). Early stopping — but when? In *Neural Networks: Tricks of the Trade*, Springer, 55–69.
- [6] Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2017). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18, 1–52.
- [7] Jamieson, K. and Talwalkar, A. (2016). Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, 240–248.
- [8] Domhan, T., Springenberg, J.T., and Hutter, F. (2015). Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence*, 3460–3468.
- [9] Baker, B., Gupta, O., Naik, N., and Raskar, R. (2017). Designing neural network architectures using reinforcement learning. In *International Conference on Learning Representations*.
- [10] Mellor, J., Turner, J., Storkey, A., and Crowley, E.J. (2021). Neural architecture search without training. In *Proceedings of the 38th International Conference on Machine Learning*, 7588–7598.
- [11] Abdelfattah, M.S., Mehrotra, A., Dudziak, L., and Lane, N.D. (2021). Zero-cost proxies for lightweight NAS. In *International Conference on Learning Representations*.
- [12] Zoph, B. and Le, Q.V. (2017). Neural architecture search with reinforcement learning. In *International Conference on Learning Representations*.

- [13] Real, E., Aggarwal, A., Huang, Y., and Le, Q.V. (2019). Regularized evolution for image classifier architecture search. In *AAAI Conference on Artificial Intelligence*.
- [14] Liu, H., Simonyan, K., and Yang, Y. (2019). DARTS: Differentiable architecture search. In *International Conference on Learning Representations*.
- [15] Barnicle, S. (2025). Task-Identity: Detecting Behavioral Drift Beyond Accuracy and Embedding Similarity. Provisional Patent Application #63/906,072.
- [16] Barnicle, S. (2025). Method and System for Predicting Neural Network Training Efficiency from Early Behavioral Identity Formation. Provisional Patent Application #63/914,409.
- [17] Barnicle, S. (2026). Predicting Neural Network Architecture Viability After One Training Epoch via an Identity Deficit Threshold. Zenodo.

Patent Disclosures

The methods described in this paper are the subject of U.S. Provisional Patent Application No. 63/938,279 (filed December 11, 2025). No license to implement or commercialize the described methods is granted by this publication. All rights reserved.